

TUGAS MANDIRI MODUL 08

HILL CLIMBING

Nama : Nuevalen Refitra Alswando
NIM : 10307243008
Kelas : IF-04-01

SOAL 1

Sebuah perusahaan travel ingin membuat sistem optimasi rute perjalanan wisata menggunakan algoritma Hill Climbing. Setiap kota direpresentasikan sebagai titik koordinat (x, y), dan jarak antar kota dihitung menggunakan jarak Euclidean.

Program harus dapat menentukan rute perjalanan yang menghasilkan total jarak minimum dengan mengunjungi setiap kota tepat satu kali (Traveling Salesman Problem sederhana).

KETENTUAN PROGRAM:

1. Representasikan kota dalam bentuk list koordinat, contoh:

```
cities = [(0,0), (2,3), (5,1), ...]
```

2. Buat fungsi untuk menghitung jarak antar kota (Euclidean Distance)
3. Buat fungsi untuk menghitung total jarak dari suatu rute
4. Implementasikan algoritma Hill Climbing dengan ketentuan:
 - o Solusi awal di-generate secara acak
 - o Neighbor diperoleh dengan menukar dua posisi kota (swap)
 - o Pilih neighbor dengan nilai terbaik (jarak minimum)
5. Program harus menampilkan:
 - o Rute awal
 - o Rute terbaik hasil optimasi
 - o Total jarak sebelum dan sesudah optimasi
6. Jika tidak ditemukan solusi yang lebih baik, tampilkan pesan:
"Solusi optimal lokal telah ditemukan"

Code

```

module > mandiri > ... > ...
You, 56 minutes ago | 1 author (You)
1 import random
2 import math
3
4 cities = [(0,0), (2,5), (5,2), (8,8), (1,7), (6,1), (3,9), (7,3)]
5
6 def euclidean(c1, c2):
7     return math.sqrt((c1[0]-c2[0])**2 + (c1[1]-c2[1])**2)
8
9 def total_distance(route, cities):
10    return sum(euclidean(cities[route[i]], cities[route[(i+1) % len(route)]]) for i in range(len(route)))
11
12 initial_route = list(range(len(cities)))
13 random.shuffle(initial_route)
14
15 current_route = initial_route[:]
16 current_dist = total_distance(current_route, cities)
17
18 while True:
19     neighbors = []
20     for i in range(len(current_route)):
21         for j in range(i+1, len(current_route)):
22             n = current_route[:]
23             n[i], n[j] = n[j], n[i]
24             neighbors.append(n)
25
26     best_neighbor = min(neighbors, key=lambda r: total_distance(r, cities))
27     best_dist = total_distance(best_neighbor, cities)
28
29     if best_dist >= current_dist:
30         print("\nSolusi optimal lokal telah ditemukan")
31         break
32
33     current_route, current_dist = best_neighbor, best_dist
34
35 print(f"Rute awal      : {initial_route}")
36 print(f"Jarak awal      : {total_distance(initial_route, cities):.4f}")
37 print(f"Rute terbaik     : {current_route}")
38 print(f"Jarak terbaik    : {current_dist:.4f}")

```

Ouput

```

(.venv) PS C:\Storage\Documents\Telkom University\Semester 4\Dasar Kecerdasan Artifisial\praktikum\modul08\mandiri> python soal1.py

Solusi optimal lokal telah ditemukan
Rute awal      : [4, 7, 3, 5, 1, 2, 6, 0]
Jarak awal     : 53.3277
Rute terbaik   : [5, 7, 3, 6, 4, 1, 0, 2]
Jarak terbaik  : 29.6831

```

SOAL 2

Sebuah perusahaan ingin mengoptimalkan penjadwalan tugas karyawan menggunakan algoritma Hill Climbing. Setiap solusi merepresentasikan urutan pengerjaan tugas, dan setiap urutan memiliki nilai "cost" berdasarkan waktu penyelesaian total.

Program harus membantu menemukan urutan tugas dengan waktu total paling minimum.

KETENTUAN PROGRAM:

1. Representasikan tugas dalam bentuk list, contoh:

```
tasks = ['A', 'B', 'C', 'D', 'E']
```

2. Tentukan sendiri fungsi evaluasi (cost function), misalnya:

- o Berdasarkan waktu pengerjaan tiap tugas
- o Atau kombinasi waktu dan prioritas

3. Implementasikan Hill Climbing dengan ketentuan:

- o Solusi awal berupa urutan acak tugas
- o Neighbor dihasilkan dengan menukar dua tugas
- o Pilih solusi dengan cost lebih kecil

4. Tambahkan fitur:

- o Menampilkan setiap iterasi perubahan solusi
- o Menampilkan nilai cost pada setiap iterasi

5. Output program harus mencakup:

- o Solusi awal
- o Proses iterasi (perubahan solusi)
- o Solusi terbaik
- o Total cost akhir

6. Analisis:

Jelaskan apakah solusi yang diperoleh merupakan **global optimum** atau hanya **local optimum**, serta alasan yang mendukung.

Code

```
1 import random
2
3 tasks = ['A', 'B', 'C', 'D', 'E']
4 durations = {'A': 4, 'B': 7, 'C': 3, 'D': 5, 'E': 6}
5
6 def calculate_cost(schedule):
7     total, current_time = 0, 0
8     for t in schedule:
9         current_time += durations[t]
10        total += current_time
11    return total
12
13 initial_schedule = tasks[:]
14 random.shuffle(initial_schedule)
15
16 current_schedule = initial_schedule[:]
17 current_cost = calculate_cost(current_schedule)
18 iteration = 1
19
20 print(f"♦ Solusi awal: {initial_schedule} | Cost: {current_cost}")
21 print("-" * 45)
22
23 while True:
24     neighbors = []
25     for i in range(len(current_schedule)):
26         for j in range(i+1, len(current_schedule)):
27             n = current_schedule[:]
28             n[i], n[j] = n[j], n[i]
29             neighbors.append(n)
30
31     best_neighbor = min(neighbors, key=calculate_cost)
32     best_cost = calculate_cost(best_neighbor)
33
34     if best_cost < current_cost:
35         current_schedule, current_cost = best_neighbor, best_cost
36         print(f"Iterasi {iteration}: {current_schedule} | Cost: {current_cost}")
37         iteration += 1
38     else:
39         print(f"\nIterasi {iteration}: Tidak ada cost lebih kecil. Proses berhenti.")
40         break
41
42 print(f"Solusi terbaik : {current_schedule}")
43 print(f"Total cost      : {current_cost}")
44
```

Output

```
(.venv) PS C:\Storage\Documents\Telkom University\Semester 4\Dasar Kecerdasan Artifisial\praktikum\modul08\mandiri> python soal2.py
Solusi awal: ['C', 'B', 'A', 'E', 'D'] | Cost: 72
-----
Iterasi 1: ['C', 'D', 'A', 'E', 'B'] | Cost: 66
Iterasi 2: ['C', 'A', 'D', 'E', 'B'] | Cost: 65
Iterasi 3: Tidak ada cost lebih kecil. Proses berhenti.
Solusi terbaik : ['C', 'A', 'D', 'E', 'B']
Total cost      : 65
```

Penjelasan

Berdasarkan hasil eksekusi yang saya lakukan, saya menyimpulkan bahwa solusi

penjadwalan ini merupakan local optimum, bukan global optimum. Hal ini disebabkan oleh sifat algoritma Hill Climbing yang greedy, yakni hanya menerima tetangga yang langsung menurunkan cost. Ketika seluruh pertukaran dua tugas tidak lagi memberikan perbaikan, algoritma langsung berhenti tanpa dapat keluar dari jebakan optimum lokal karena tidak memiliki mekanisme untuk menerima solusi buruk sementara atau eksplorasi acak. Oleh karena itu, tanpa strategi tambahan seperti Simulated Annealing, hasil yang saya peroleh hanyalah solusi terbaik di sekitar titik awal, bukan yang paling optimal secara menyeluruh.